

The whole process ORB WEAVER

Think of the system as three separate things:

ORB WEAVER
= manufacturer

website_orb_final
= golden cloneable template

customer Website ORB
= finished manufactured product

Orb Weaver already has a real scan center / full-scan workflow and its own canonical vault infrastructure.

What we are doing now is upgrading the **manufacturer** so it can populate the new intelligence system in the template.

Stage 1 — Finish the golden template

This is what we have been working on in:

`C:\Users\bryan\Desktop\website_orb_final`

It now contains the new Kimi system:

```
Orb_Vault_System/  
├── orb_vault_skg/  
│   ├── vault/           ← runtime code  
│   └── vaults/  
│       ├── A_Priori_Vault/  
│       └── A_Posteriori_Vault/
```

The coordinator is already wired so the runtime knows where those two physical vaults are.

The golden template should **not contain customer-specific knowledge**. It should contain the machinery and empty/default vault containers.

So this stage is essentially done structurally.

Stage 2 — Put the same vault/SKG machinery into Orb Weaver's ORB-building side

This is the **next thing we do**.

Not a rescan yet.

Orb Weaver needs access to the same schema/runtime expectations so it knows what it is manufacturing.

But there is an important distinction:

Orb Weaver's own canonical `vault_system/` is still its persistent builder storage authority. We should not casually replace that. Its own documentation says it is the single authoritative storage root for the Orb Weaver repository.

So what we need to establish is an **ORB manufacturing/template location inside Orb Weaver** for `orb_vault_skg`, rather than confusing that with Orb Weaver's own operational memory.

That location then becomes the source copied into each manufactured Website ORB.

Stage 3 — Add the things Orb Weaver does NOT yet manufacture

This is the major missing work.

The new A Priori Vault expects four core compiled artifacts:

catalog.json
ontology.json / site.skg
qa.json
policies.json

Orb Weaver already has site crawling, Site World generation, owner scanning, pointer mapping and related scan infrastructure. The repo has a Scan Center and Web Weave implementation for that existing scanning pipeline.

What appears to be missing

Based on the current repo inspection and code search, I do **not** find an existing production catalog compiler producing the new `catalog.json` contract.

So we need to add:

1. **Catalog extraction/compiler**
2. **Ontology/SKG compiler for the new vault contract**
3. **QA compiler**
4. **Policy compiler**
5. **Packaging injection step**

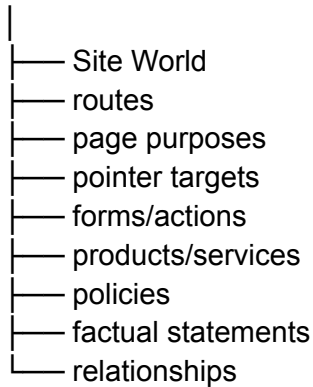
Those are not five unrelated scanners.

They should all consume the **same fresh Full Scan evidence**.

Stage 4 — What the Full Scan should eventually produce

When Orb Weaver scans a site, it should collect the site once and derive several products from that scan:

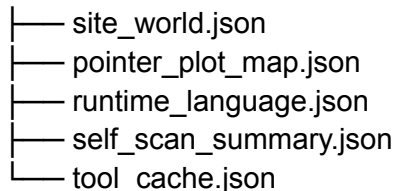
FULL WEBSITE WEAVE



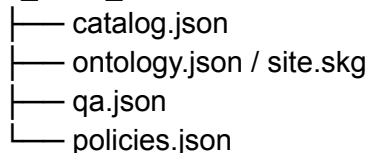
COMPILE STAGE

Then compilation produces:

compiled_orb/



A_Priori_Vault/



Those two groups are related but have different responsibilities.

compiled_orb describes the **operating environment**.

A_Priori_Vault describes the **settled knowledge about that environment**.

Stage 5 — How catalog and pointer become connected

This part is critical.

We should not merely extract:

Product:
Orb Weaver Enhanced
Price:
\$1,488

We also want to retain correspondence to the page object:

product_id:
product_123

route:
/marketplace

pointer_target:
target_xyz

source_url:
...

verified:
true

That creates the bridge:

semantic identity
+
physical website identity

So later:

“How much is Enhanced ORBS?”

The vault answers the price.

Then:

“Show me.”

The runtime already knows which route and verified pointer target correspond to that product.

That is how vault intelligence joins **voice + motion + pointer**.

Stage 6 — Full fresh rescan of ALL Orb Weaver

Only after the compiler additions are in place do we run the scan.

And you are correct: **we scan all of Orb Weaver, not merely the Marketplace.**

Why?

Because the Marketplace gives us products/prices.

Other pages give us:

- business/site identity
- ORB explanations
- feature relationships
- policies
- contact information
- support information
- routes
- calls to action
- forms
- navigation
- pointer targets
- QA-worthy facts

So the Marketplace contributes to the catalog, while the rest of Orb Weaver contributes to ontology, QA, policies, Site World and pointer knowledge.

One scan becomes the authoritative fresh snapshot.

Stage 7 — Verify the compiled vault BEFORE manufacturing

After the rescan, we inspect:

A_Priori_Vault/catalog.json
A_Priori_Vault/ontology.json
A_Priori_Vault/qa.json
A_Priori_Vault/policies.json

We make sure things like:

product names correct
prices correct
routes correct
relationships correct
no invented policies
no duplicate products
pointer correspondence valid

This is where owner verification matters.

We do **not** immediately ship whatever the crawler guessed.

Stage 8 — Clone `website_orb_final`

Then Orb Weaver takes the pristine golden template:

```
website_orb_final
```

and creates a fresh customer build:

```
website_orb_build_<customer>
```

Then it injects:

```
fresh compiled_orb data
```

```
fresh A_Priori_Vault data
```

The A Posteriori vault starts clean:

```
A_Posteriori_Vault/
```

```
└── ledger/
```

because that customer's ORB hasn't learned anything yet.

Stage 9 — Build the customer installer

Orb Weaver's pack-generator area already exists in the repo, so this is where the final manufacturing integration ultimately belongs.

It should produce something like:

```
Customer_ORB_Install.zip
```

containing:

```
runtime
```

```
frontend
```

```
backend
```

```
compiled_orb
```

```
Orb_Vault_System
```

```
voice integration
```

```
configuration
```

```
installer/startup
```

Then we install **that**, not the template itself.

Stage 10 — First real acceptance test

For the first manufactured test ORB, using Orb Weaver itself gives us the strongest validation.

We test these in sequence:

Knowledge test

“How much is [Marketplace product]?”

Expected:

A Priori

- catalog
- direct answer
- TTS
- no model

SKG test

“What's the difference between Basic and Enhanced?”

Expected:

ontology/SKG

- related product facts
- answer

Policy test

Ask something actually present in the scanned site.

Navigation test

“Show me Enhanced ORBS.”

Expected:

semantic product

- route/target correspondence
- pointer verification
- ORB moves
- MORB/pointer guides
- voice explains

Unknown test

Ask something the site does not know.

Expected:

A Priori miss

→ A Posteriori miss

→ TPC/model fallback

Learning test

A successful model-resolved interaction gets recorded into A Posteriori as candidate experience.

Then verification/reinforcement/promotion/pruning govern what happens to it.

What is missing right now

This is the checklist I would keep in front of us:

Component	Status
Website ORB golden template	Present
A Priori runtime	Present
A Posteriori runtime	Present
Physical A Priori vault	Present
Physical A Posteriori vault	Present
Coordinator path wiring	Present
Existing Site World scan	Present
Existing pointer scan/map	Present
Existing Website ORB voice/runtime concepts	Present
Existing packaging infrastructure	Present
Catalog compiler	Missing / must build
New vault ontology/SKG compiler	Must verify/build

QA compiler	Must build/verify
Policy compiler	Must build/verify
Compiler → A Priori injection	Missing
Product/entity → pointer correspondence	Must wire
Golden template clone injection	Must wire/verify
Installer acceptance test	Not yet performed
Full fresh Orb Weaver rescan	Waiting until above is ready

That is our roadmap.

And the immediate next step is only one thing

Do not rescan yet.

We go into `Orb_Weaver` and establish the correct place for the new `orb_vault_skg` manufacturing package inside the ORB-building architecture.

Once that is correctly placed, we build the **catalog compiler first**.

Then SKG/QA/policy compilation.

Then we perform the full Orb Weaver rescan.

That sequence keeps us from getting lost or doing the same scan twice.